

MySQL



第一章：数据库基本概念

1. 从“记事儿”到“用数据”

在没有计算机的时代，人们记账靠账本，管理员工信息靠花名册。这些数据被记录在纸上，查找时得一页一页翻。后来有了文件系统，我们可以把数据存成文本文件、Excel 表格。比如用 Python 写个小脚本，把用户信息存在 `users.txt` 里：

```
python
1 # 文件存储示例（仅示意，非课堂执行代码）
2 with open('users.txt', 'a') as f:
3     f.write('张三,28,北京\n')
```

刚开始一切顺利，直到有一天需求变了：老板想找出所有年龄大于 30 且在北京的用户。你得遍历整个文件，逐行解析，自己写筛选逻辑。更麻烦的是，如果多个程序同时修改这个文件，数据就可能损坏或丢失；如果数据量达到百万级别，文件读取的效率会让你崩溃。

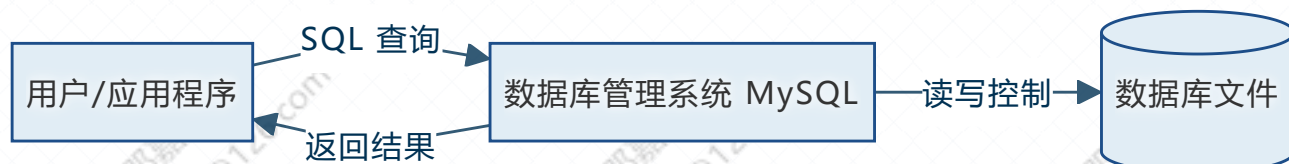
我们需要一种**专门管理数据的工具**，它能高效地**存储、查询、修改、保护共享的数据**，这就是数据库。

2. 数据库与数据库管理系统

数据库 (Database, DB) 可以通俗地理解为“存放数据的仓库”，但这个仓库不只是物理存储，更重要的是按照一定的**数据结构**来组织数据，让访问变得简单。

数据库管理系统 (Database Management System, DBMS) 则是管理这个仓库的软件。我们和数据库打交道，都是通过 DBMS 提供的接口。MySQL、Oracle、PostgreSQL、SQLite 等都属于关系型数据库管理系统 (RDBMS)。

下图展示了用户、应用、DBMS 和数据库之间的关系：



一图顶万言：应用程序不直接操作磁盘上的数据文件，而是通过 DBMS 提供的统一接口来实现。这样做的好处是隔离了物理存储细节，保证了数据的安全、并发和一致性。

3. 数据库模型简史

历史上出现过多种数据库模型，我们只做简要了解，重点放在今天的主角——关系模型上。

- **层次模型：**数据组织成树形结构，像公司的组织架构。查找一个员工的上级很容易，但查找“所有参与过某个项目的员工”就很难。
- **网状模型：**比层次模型灵活，数据像一张网，节点之间可以有多重联系。复杂度较高，编程门槛高。
- **关系模型：**1970 年由 IBM 研究员 E.F.Codd 提出。核心思想是把数据看成一张张**二维表**（由行和列组成），通过**关系**将表关联起来。这个模型基于严格的数学集合论，结构简单、清晰，成为商业数据库的事实标准。

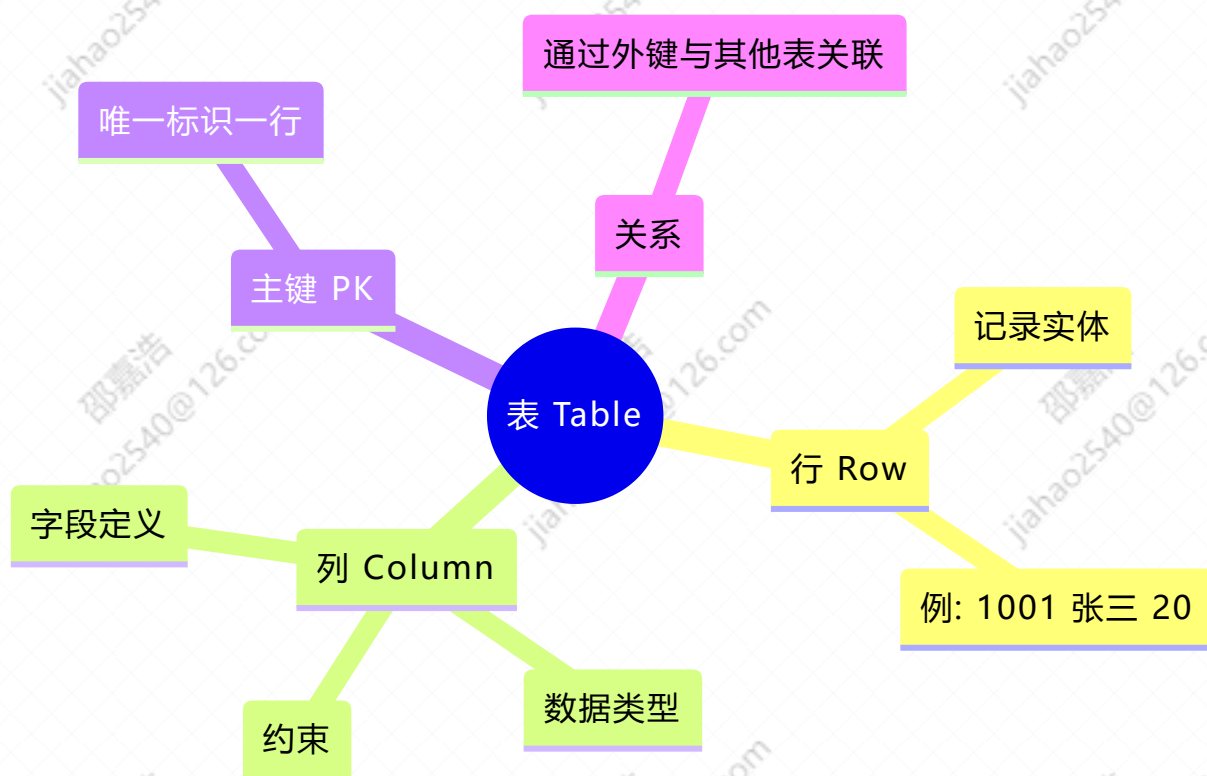
我们后续所有的内容，包括 MySQL，都建立在关系模型之上。

4. 关系型数据库的核心概念

这是本章的**重中之重**。我们把数据想象成一个 Excel 工作簿：每个 Sheet 就是一张表，每一列是**字段**，每一行是**记录**。

4.1 表 (Table)、行 (Row)、列 (Column)

表由行和列组成。每一列都有一个名字（字段名）和数据类型，比如“姓名”是文本，“年龄”是整数。每一行代表一个完整的实体记录，比如一个学生。



4.2 主键 (Primary Key)

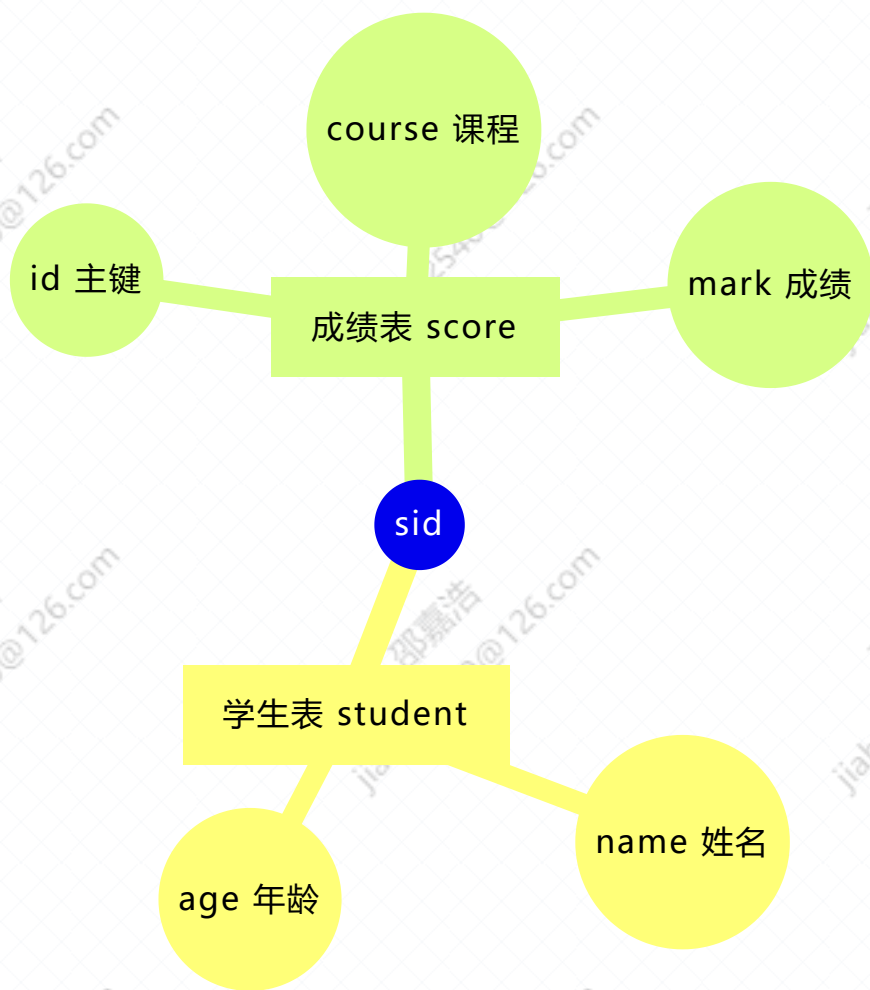
每一行数据都需要一个**唯一标识**，就像每个人的身份证号。在设计表时，我们会指定一列（或多列的组合）作为主键。主键的特性是**非空且唯一**。

例如，`student` 表中，`sid` 就是天然的主键。有了主键，我们就能快速定位到某一行数据，后续建立表与表之间的关联也全靠它。

4.3 外键 (Foreign Key)

外键用来表示**表和表之间的关联关系**。假设我们还有一个 `score` 成绩表，它记录每个学生各科目的成绩。那么在 `score` 表中，需要一列 `sid` 表示这个成绩属于哪个学生。这个 `sid` 就是外键，它引用 `student` 表的主键。

这样，我们就可以从“成绩”自然追溯到“学生”的详细信息。这种用关系把表连接起来的能力，是关系型数据库的强大所在。



4.4 约束 (Constraint)

约束是加在表列上的规则，用来保证数据的完整性和准确性。可以理解为数据的“安检通道”——不符合规则的数据会被拒绝入库。

- ① **主键约束 (PRIMARY KEY)**：唯一标识一行，非空且唯一，每表只能有一个。
- ② **外键约束 (FOREIGN KEY)**：建立表间关联，确保引用完整性。
- ③ **唯一约束 (UNIQUE)**：值不能重复，但允许 NULL。
- ④ **非空约束 (NOT NULL)**：必须有值，不能留空。
- ⑤ **默认值约束 (DEFAULT)**：未指定值时自动填充预设值。

5. SQL 语言

SQL (Structured Query Language, 结构化查询语言) 是操作关系型数据库的标准语言。无论是 MySQL、Oracle 还是 PostgreSQL，基本 SQL 语法都高度一致。

SQL 按功能可以分为四个子类：

- **DQL (数据查询语言)**：SELECT 一 查询数据，最常使用。

- **DML (数据操作语言):** INSERT、UPDATE、DELETE — 对表中的数据进行增、改、删。
- **DDL (数据定义语言):** CREATE、ALTER、DROP、TRUNCATE — 定义和修改表结构。
- **DCL (数据控制语言):** GRANT、REVOKE — 控制用户访问权限。

第二章：MySQL 简介

1. 认识 MySQL

1.1 MySQL 是什么？

MySQL 是一个**开源的关系型数据库管理系统 (RDBMS)**，由瑞典 MySQL AB 公司开发，后来被 Oracle 公司收购。它使用 SQL (结构化查询语言) 来管理数据，具有性能高、可靠性强、易于使用等特点，目前是全球最流行的开源数据库之一。

如果把数据比作货物，数据库就是仓库，而 MySQL 就是那个管理仓库的专业团队——负责货物的入库、出库、盘点和安全。它支持多种操作系统 (Linux、Windows、macOS)，被广泛应用于 Web 应用、数据仓库、日志系统等场景。

1.2 MySQL 的主要特点

特点	说明
开源免费	社区版遵循 GPL 协议，可以免费使用和修改
高性能	优化了查询速度和并发处理能力，适合高并发读写
跨平台	支持主流操作系统，部署灵活
多种存储引擎	支持 InnoDB、MyISAM 等多种存储引擎，可根据场景选择
安全性强	提供用户权限管理、SSL 加密连接等安全机制
社区活跃	拥有庞大的用户社区和丰富的文档资源

1.3 MySQL 的发展历史与版本

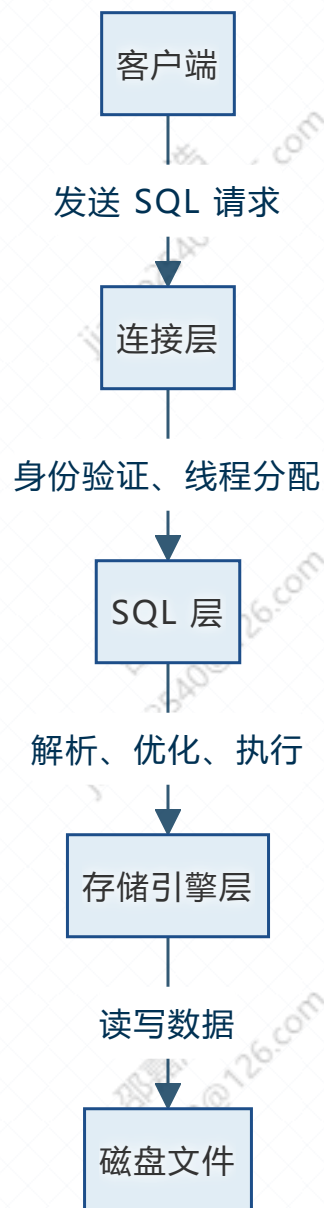
MySQL 的发展历程中有几个关键节点：

- **1995 年**：MySQL 的第一个内部版本发布。
- **2000 年**：MySQL 3.23 发布，开始支持 InnoDB 事务型存储引擎。
- **2005 年**：MySQL 5.0 发布，引入存储过程、视图、游标等高级特性。
- **2008 年**：Sun Microsystems 收购 MySQL AB。
- **2010 年**：Oracle 收购 Sun，MySQL 随之归属 Oracle。
- **2015 年**：**MySQL 5.7** 发布，成为里程碑版本，新增 JSON 原生支持、优化器改进、性能大幅提升，至今仍有大量生产环境运行。
- **2018 年**：MySQL 8.0 发布，带来窗口函数、CTE、原子 DDL 等重大更新。

本课程统一使用 **MySQL 5.6** 版本。它是久经考验的稳定版本，语法与大多数老系统兼容，同时也能平滑过渡到 8.x 新特性。

2. MySQL 的体系架构

理解 MySQL 的内部结构，有助于我们更好地理解它是如何处理一条 SQL 语句的。MySQL 采用 **模块化架构**，大致可以分为以下几层：



各层职责说明：

- **连接层**：负责与客户端建立连接，验证用户身份，维持会话。每个客户端连接会对应一个线程。
- **SQL 层**：这是 MySQL 的核心，包括：
 - **查询解析器**：检查 SQL 语法是否正确，生成解析树。
 - **查询优化器**：为查询选择最高效的执行计划（例如决定使用哪个索引、表的连接顺序）。
 - **执行器**：根据执行计划调用存储引擎的接口完成操作。
- **存储引擎层**：负责数据的物理存储和读取。MySQL 支持多种存储引擎，以插件形式管理，最常用的是 **InnoDB**（默认引擎），它支持事务、行级锁、外键等。

- **磁盘文件**：最终数据以文件形式存放，包括表结构定义（`.frm`）、数据和索引（`InnoDB` 下通常为 `.ibd` 文件）。

3. MySQL 的连接方式

要操作 MySQL，我们首先需要连接到服务器。

3.1 使用 mysql 命令行客户端

`mysql` 是 MySQL 自带的交互式命令行工具，安装 MySQL Server 后会同时拥有它。连接的基本语法

```
mysql -h 主机地址 -P 端口 -u 用户名 -p
```

参数说明：

- `-h`：服务器主机地址，本机连接可用 `11..0.0.1` 或 `localhost`。
- `-P`：端口号，MySQL 默认端口为 `3306`（注意是大写 P）。
- `-u`：登录用户名，这里用最高权限的 `root` 用户做演示。
- `-p`：密码参数（小写 p），回车后会提示输入密码，输入时屏幕不会显示字符，这是安全设计。

输入正确密码后，终端会出现 `mysql>` 提示符，表示已经成功进入 MySQL 交互环境。此时就可以执行 SQL 语句了。

本地连接测试示例：

```
-- 查看 MySQL 版本
SELECT VERSION();

-- 查看当前连接的用户
SELECT USER();

-- 查看当前日期和时间
SELECT NOW();
```

3.2 命令行操作的不便之处

虽然命令行客户端功能完整，但在实际工作中存在一些明显的不便：

- **界面不直观**：所有信息以纯文本形式展示，当查询结果列数较多、数据量较大时，排版容易混乱，难以快速浏览。
- **无语法高亮与自动补全**：输入 SQL 时没有关键字高亮，表名、列名的自动提示支持有限，容易写错。
- **管理多库多表效率低**：在命令行中切换数据库、浏览表结构、查看数据都需要逐条输入命令，缺乏全局可视化的导航。
- **历史记录与结果导出不便**：虽然命令行有历史功能，但跨会话复用、保存查询结果等操作远不如图形化工具方便。

基于这些痛点，许多开发者和数据分析师会选择使用**图形化客户端工具**来连接和管理 MySQL。

3.3 使用 DataGrip 等图形化工具

DataGrip 是 JetBrains 公司推出的一款专业数据库管理工具，支持 MySQL、PostgreSQL、Oracle 等多种数据库。它提供了以下优势：

- **可视化界面**：数据库、表、视图以树形结构展示，一目了然。
- **智能编辑**：SQL 编辑器支持语法高亮、自动补全、错误提示。
- **结果集管理**：查询结果以表格形式呈现，支持排序、过滤、导出。
- **多会话管理**：可以同时打开多个连接，方便对比不同环境的数据。

除了 DataGrip，常见的同类工具还有 **MySQL Workbench**（官方免费）、**DBeaver**（开源免费）、**Navicat** 等，你可以根据自己的喜好选择。本课程后续的演示会以 DataGrip 为主，因为它界面简洁、功能强大，且和 IntelliJ IDEA、PyCharm 等开发工具风格一致，能降低学习成本。

3.4 创建远程管理账号并开启远程访问

图形化工具通常安装在本地笔记本电脑上，而 MySQL 运行在服务器（或虚拟机）上，因此需要**远程连接**。直接开放 root 用户远程权限容易出现密码混乱、多 host 冲突等问题。我们采用更统一、更安全的做法：**创建一个全新的远程管理账号，并赋予最高权限**。

首先在服务器本地使用命令行登录 MySQL（使用 root 账号）：

bash

```
1 | mysql -h 127.0.0.1 -P 3306 -u root -p
```

进入 `mysql>` 交互界面后，执行以下命令：

sql

```
1 | -- 创建名为 admin 的新用户，允许从任意 IP 远程连接，并设置密码
2 | GRANT ALL PRIVILEGES ON *.* TO 'admin'@'%' IDENTIFIED BY
   | 'admin' WITH GRANT OPTION;
3 | -- 刷新权限，使设置立即生效
4 | FLUSH PRIVILEGES;
```

命令解读：

- `'admin'@'%'`：用户名是 `admin`，`%` 表示允许任何 IP 连接。你可以将其改为特定 IP 地址以限制访问。
- `IDENTIFIED BY 'admin'`：设定密码。
- `ALL PRIVILEGES ON *.*`：授予所有数据库的全部权限。
- `WITH GRANT OPTION`：允许该用户将自身权限再授予其他用户。

验证新用户是否创建成功：

sql

```
1 | SELECT user, host FROM mysql.user WHERE user = 'admin';
```

应出现类似以下记录：

text

```
1 | +-----+-----+
2 | | user  | host  |
3 | +-----+-----+
4 | | admin | %    |
5 | +-----+-----+
```

现在，这个 `admin` 账号已经具备了远程连接的所有权限。

第三章：数据定义语言 DDL

1. DDL 概述

1.1 什么是 DDL?

SQL 按功能分为四大类：DQL（查询）、DML（操作数据）、DDL（定义结构）、DCL（权限控制）。DDL 处理的不是表内的数据，而是表本身、数据库本身。它包含以下核心动词：

动词	作用	操作对象
CREATE	创建	数据库、表
ALTER	修改	表结构
DROP	删除	数据库、表
TRUNCATE	清空表数据	表

1.2 DDL 的执行特点

- 自动提交**：DDL 语句执行后，会立刻生效，并且无法“回滚”（不可逆）。
- 影响结构**：直接修改数据字典（元数据），操作成本较高，生产环境需谨慎。
- 语法统一**：MySQL 中 DDL 语法高度标准化，学习一次可适用于大多数关系型数据库。

2. 数据库的创建与删除

在实际工作中，一个项目通常拥有独立的数据库。我们首先要学会**管理数据库本身**。

2.1 创建数据库

基础语法：

sql

```
1 CREATE DATABASE [IF NOT EXISTS] 数据库名
2     [DEFAULT CHARACTER SET 字符集]
3     [DEFAULT COLLATE 校验规则];
```

- **IF NOT EXISTS**：可选，避免因数据库已存在而报错。
- **CHARACTER SET**：指定字符编码，推荐使用 **utf8mb4**（支持完整 Unicode，包括 emoji）。
- **COLLATE**：指定字符比较规则，通常 **utf8mb4_unicode_ci**（不区分大小写）。

示例代码：

sql

```
1 -- 创建一个名为 mydb 的数据库
2 CREATE DATABASE IF NOT EXISTS mydb
3     DEFAULT CHARACTER SET utf8mb4
4     DEFAULT COLLATE utf8mb4_unicode_ci;
```

验证：

sql

```
1 -- 查看所有数据库
2 SHOW DATABASES;
3
4 -- 查看创建语句（可以看到字符集等信息）
5 SHOW CREATE DATABASE mydb;
```

2.2 删除数据库

sql

```
1 DROP DATABASE [IF EXISTS] 数据库名;
```

- **IF EXISTS**：防止数据库不存在时出错。
- **极其危险**：会删除该库下所有表和数据，无法恢复。

示例：

sql

```
1 DROP DATABASE IF EXISTS mydb;
```

执行后再次执行 `SHOW DATABASES;`，`mydb` 将消失。

3. 表的创建 (CREATE TABLE)

表是数据存储的基本单元。创建一个表需要定义表名、各列（字段）及其数据类型、约束。

3.1 基本语法

sql

```
1 CREATE TABLE [IF NOT EXISTS] 表名 (  
2     列名1 数据类型 [约束],  
3     列名2 数据类型 [约束],  
4     ...  
5     [表级约束]  
6 ) [ENGINE=引擎] [CHARSET=字符集] [COMMENT='注释'];
```

3.2 数据类型速查

MySQL 的数据类型非常丰富，我们在本章先快速建立印象，下一章会重点展开。这里列举最常用的几类：

- **整数**： `INT`、`TINYINT`、`BIGINT` 等。
- **小数**： `DECIMAL(M,D)`（定点数，适合金额）、`FLOAT`、`DOUBLE`。
- **字符串**： `VARCHAR(n)`（可变长，最大 65535 字节）、`CHAR(n)`（定长）、`TEXT`（长文本）。
- **日期时间**： `DATE`（日期）、`TIME`（时间）、`DATETIME`（日期时间）、`TIMESTAMP`（时间戳）。
- **二进制**： `BLOB`（二进制大对象）。

3.3 字段约束回顾

上一章我们学过五种约束，这里结合建表再次巩固：

约束	关键字	说明
主键	PRIMARY KEY	非空唯一，一张表一个
外键	FOREIGN KEY	关联另一张表的主键
唯一	UNIQUE	值不可重复，可多个 NULL
非空	NOT NULL	必须提供值
默认值	DEFAULT	未指定时自动填充

此外还有 `AUTO_INCREMENT` 自增属性，通常用于整数主键。

3.4 动手创建学生表与成绩表

sql

```
1  -- 先切换到目标库（如果还没有库，先创建）
2  CREATE DATABASE IF NOT EXISTS mydb
3      DEFAULT CHARACTER SET utf8mb4
4      DEFAULT COLLATE utf8mb4_unicode_ci;
5  USE mydb;
6
7  -- 创建学生表
8  CREATE TABLE IF NOT EXISTS student (
9      sid INT PRIMARY KEY COMMENT '学号，主键',
10     name VARCHAR(50) NOT NULL COMMENT '姓名',
11     age TINYINT UNSIGNED COMMENT '年龄 0-255',
12     email VARCHAR(100) UNIQUE COMMENT '邮箱',
13     created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14     COMMENT '创建时间'
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='学生信息
16 -- 创建成绩表
```

```

17 CREATE TABLE IF NOT EXISTS score (
18     id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键',
19     sid INT NOT NULL COMMENT '学生学号',
20     course VARCHAR(50) NOT NULL COMMENT '课程名称',
21     mark DECIMAL(4,1) COMMENT '成绩',
22     CONSTRAINT fk_score_sid FOREIGN KEY (sid) REFERENCES
        student(sid)
23     ON DELETE CASCADE ON UPDATE CASCADE
24 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='成绩表';

```

语句解读：

- `COMMENT '...'`：为列或表添加注释，便于维护。
- `TINYINT UNSIGNED`：范围 0~255，适合年龄。
- `TIMESTAMP DEFAULT CURRENT_TIMESTAMP`：记录创建时间，插入时自动填充当前时间。
- 成绩表中外键 `ON DELETE CASCADE`：删除学生时自动删除其成绩，保证引用完整性。
- 所有表指定了 `ENGINE=InnoDB` 和字符集，养成良好习惯。

验证：

```

-- 查看当前库有哪些表
1 SHOW TABLES;
2
3
4 -- 查看表结构
5 DESC student;
6 DESC score;
7
8 -- 查看完整建表语句
9 SHOW CREATE TABLE student;

```

3.5 建表时的常用选项

- `IF NOT EXISTS`：避免重复创建报错。

- **AUTO_INCREMENT**: 只能用于整数列, 且通常是主键。默认从 1 开始, 步长为 1。
- **ENGINE**: 建议始终显式指定 InnoDB, 它支持行级锁、外键约束, 数据安全性高, 是 MySQL 5.6.45 的默认引擎。
- **CHARSET**: 表级字符集, 可覆盖数据库默认值。

4. 表的修改 (ALTER TABLE)

表结构建成后, 需求难免变化。 **ALTER TABLE** 可以添加、删除、修改列和约束。

4.1 添加列

```
1 | ALTER TABLE 表名 ADD 列名 数据类型 [约束] [FIRST | AFTER 已有列名];
```

示例:

```
1 | -- 在 student 表最后添加一列 phone
2 | ALTER TABLE student ADD phone VARCHAR(20) COMMENT '电话号码';
3 |
4 | -- 在 name 列后面添加 gender 列
5 | ALTER TABLE student ADD gender CHAR(1) COMMENT '性别' AFTER name;
```

验证:

```
1 | DESC student;
```

4.2 修改列

可以修改列的数据类型、约束或名称。

sql

```
1 ALTER TABLE 表名 MODIFY 列名 新数据类型 [约束];  
2 ALTER TABLE 表名 CHANGE 旧列名 新列名 新数据类型 [约束];
```

区别：

- **MODIFY** 不能改列名，只改定义。
- **CHANGE** 可以改列名和定义。

示例：

sql

```
1 -- 修改 phone 列长度为 30，添加非空约束（注意原来数据可能为 NULL，需先  
   处理）  
2 ALTER TABLE student MODIFY phone VARCHAR(30) NOT NULL;  
3  
4 -- 将 gender 列名改为 sex，类型不变  
5 ALTER TABLE student CHANGE gender sex CHAR(1) COMMENT '性别';
```

注意：修改数据类型可能造成数据截断或精度丢失，需小心。生产环境修改大表时可能需要考虑锁表时间。

4.3 删除列

sql

```
1 ALTER TABLE 表名 DROP 列名;
```

示例：

sql

```
1 ALTER TABLE student DROP phone;
```

一旦删除，数据不可恢复。

4.4 添加/删除约束

可以单独添加或删除约束。

添加主键：

```
1  -- 假设建表时忘了设置主键
2  ALTER TABLE student ADD PRIMARY KEY (sid);
```

添加外键：

```
1  -- 如果当初没建外键，后期可以加上
2  ALTER TABLE score ADD CONSTRAINT fk_sid FOREIGN KEY (sid)
   REFERENCES student(sid);
```

删除外键：

```
1  ALTER TABLE score DROP FOREIGN KEY fk_sid;
```

删除主键：

```
1  ALTER TABLE student DROP PRIMARY KEY;
```

注意：若主键有自增属性，直接删主键会报错，需先 **MODIFY** 去掉自增。

4.5 修改表名

```
1  ALTER TABLE 旧表名 RENAME TO 新表名;
2  -- 或
3  RENAME TABLE 旧表名 TO 新表名;
```

示例：

```
1  ALTER TABLE student RENAME TO stu;
2  RENAME TABLE stu TO student;
```

5. 表的删除与清空

5.1 删除表 (DROP TABLE)

```
1 DROP TABLE [IF EXISTS] 表名;
```

示例:

```
1 DROP TABLE IF EXISTS score;
2 DROP TABLE IF EXISTS student;
```

一旦删除, 表结构和数据全部消失, 不可恢复。

5.2 清空表数据 (TRUNCATE TABLE)

```
1 TRUNCATE [TABLE] 表名;
```

示例:

```
1 TRUNCATE TABLE score;
```

与 DELETE 对比:

特性	TRUNCATE	DELETE
类型	DDL	DML
自增计数器	重置为初始值	不重置
速度	快 (直接释放空间)	慢 (逐行删除)
操作可逆性	不可逆, 数据无法恢复	在特定条件下 (如开启日志) 可利用备份恢复



当需要快速清空整张表并重置自增计数时，用 TRUNCATE；当需要根据条件删除部分数据时，用 DELETE。

第四章：数据操作语言 DML

1. DML 概述

1.1 什么是 DML?

SQL 四大子语言中，DML 负责直接处理表中的数据行。它包含三个最常用的关键字：

关键字	作用	通俗理解
INSERT	向表中插入新数据	入库（增加记录）
UPDATE	修改表中已有数据	更新（改变记录内容）
DELETE	删除表中指定数据	出库（移除记录）



与 DDL 不同，DML 操作的是数据行，可以加上 WHERE 条件来精准定位。

1.2 DML 与 DDL 的区别

对比维度	DDL	DML
操作对象	数据库、表的结构	表中的数据行
是否可回滚	执行后立即自动提交，不可回滚	可回滚（后面会学到）
常见命令	CREATE , ALTER , DROP	INSERT , UPDATE , DELETE
影响	改变元数据，影响整体结构	改变实际数据，影响范围可控

2. 插入数据：INSERT

2.1 基本语法

最简单的插入语句：

```
1 INSERT INTO 表名 (列1, 列2, ...) VALUES (值1, 值2, ...);
```

- 列名列表与值列表必须**顺序对应**，数据类型也要匹配。
- 如果给表中的所有列都提供值，可以省略列名列表，但这样很危险：一旦表结构增加列，代码就会出错。**推荐始终显式写出列名。**

2.2 插入单行数据

我们使用上一章创建的 `student` 和 `score` 表进行演示。请先确认已经切换到目标数据库并确保表存在：

```
1 USE mydb;
```

插入一个学生：

```
1 INSERT INTO student (sid, name, age, email)
2 VALUES (1001, '张三', 20, 'zhangsan@example.com');
```

插入一条成绩：

```
1 INSERT INTO score (sid, course, mark)
2 VALUES (1001, '数学', 88.5);
```

验证数据是否插入成功：

```
1 SELECT * FROM student;
```

```
2 SELECT * FROM score;
```

2.3 插入多行数据

如果要一次性添加多个学生，可以用一条 `INSERT` 语句，用逗号分隔多个值列表：

```
1 INSERT INTO student (sid, name, age, email) VALUES
2 (1002, '李四', 22, 'lisi@example.com'),
3 (1003, '王五', 19, 'wangwu@example.com'),
4 (1004, '赵六', 21, 'zhaoliu@example.com');
```

同时插入多条成绩：

```
1 INSERT INTO score (sid, course, mark) VALUES
2 (1002, '数学', 76.5),
3 (1002, '英语', 80.0),
4 (1003, '语文', 85.0),
5 (1004, '数学', 90.0);
```

这样操作比多次单行插入效率更高，尤其当你需要导入大量数据时。

2.4 插入部分列（省略有默认值或可空的列）

如果某列有默认值，或允许为 `NULL`，你可以在 `INSERT` 时省略它。

示例：`student` 表有一个 `created_time` 列，它默认自动填充当前时间，因此插入时可以不用写：

```
1 INSERT INTO student (sid, name, age, email)
2 VALUES (1005, '孙七', 23, 'sunqi@example.com');
```

查询结果会看到 `created_time` 自动填入了当前时间戳。

注意：如果某列有 `NOT NULL` 约束且没有默认值，插入时必须提供，否则会报错。

3. 修改数据：UPDATE

3.1 基本语法

```
1 UPDATE 表名
2 SET 列1 = 新值1, 列2 = 新值2, ...
3 [WHERE 条件];
```

- **WHERE 子句极其重要**：如果没有 **WHERE**，**UPDATE** 会修改表中所有行！这不是玩笑，生产数据库一不小心就会造成灾难。
- **SET** 后面可以同时更新多个列，用逗号分隔。

3.2 带条件更新单列

假设我们要将学号 1001 的张三的年龄从 20 改为 21：

```
1 UPDATE student
2 SET age = 21
3 WHERE sid = 1001;
```

验证：

```
1 SELECT * FROM student WHERE sid = 1001;
```

3.3 更新多列

把李四的邮箱和年龄同时更新：

```
1 UPDATE student
2 SET age = 23, email = 'lisi_new@example.com'
3 WHERE sid = 1002;
```

3.4 使用表达式更新

可以在 `SET` 中使用算术、字符串函数等表达式。

例如，所有成绩提高 5%：

```
1 UPDATE score
2 SET mark = mark * 1.05
3 WHERE mark < 90;
```

但注意：成绩可能超出合理范围，可以用 `CASE WHEN` 或 `ROUND` 等函数控制（后面章节会讲函数）。

4. 删除数据：DELETE

4.1 基本语法

```
1 DELETE FROM 表名
2 [WHERE 条件];
```

- 没有 `WHERE` 会删除所有行！ 表结构保留，但数据全没了。
- `DELETE` 是逐行删除，效率较 `TRUNCATE` 低，但它可以带条件。

4.2 带条件删除

删除学号为 1006 的学生：

```
1 DELETE FROM student WHERE sid = 1006;
```

由于我们在创建 `score` 表时设置了 `ON DELETE CASCADE`，当删除学生时，他的成绩也会自动被删除，保持数据一致性。

4.3 清空表：DELETE vs TRUNCATE

我们在 DDL 一章已经对比过。这里再强调使用场景：

- 想保留删除操作的可能性（可撤销），或需触发某些条件时，用 **DELETE**。
- 要快速清空全表并重置自增主键，用 **TRUNCATE**。

5 实践任务

请使用 **mydb** 数据库和已有的 **student**、**score** 表，完成以下操作（全部在 MySQL 5.6.45 下测试通过）：

- 插入一个新学生（学号 1007，姓名郑十，年龄 25，邮箱 zhengshi@example.com），并给他添加两门课程成绩（课程名自拟，分数自拟）。
- 批量插入三个学生：
(1008, 陈十一, 22, chen@example.com)
(1009, 卫十二, 26, wei@example.com)
(1010, 蒋十三, 24, jiang@example.com)
- 将学号为 1008 的学生年龄改为 23，同时将邮箱改为 chen_new@example.com。
- 给所有年龄小于 24 的学生的成绩增加 5 分。
- 删除学号为 1010 的学生（注意观察成绩表的变化）。
- 删除分数低于 80 分的成绩记录。

第 五 章：数据查询语言 DQL

0. 本章初始化脚本（在 **mydb** 库中执行）

```
1  -- 使用 mydb 数据库，如果没有则先创建
2  CREATE DATABASE IF NOT EXISTS mydb
3  DEFAULT CHARACTER SET utf8mb4
```

sql

```
4      DEFAULT COLLATE utf8mb4_unicode_ci;
5  USE mydb;
6
7  -- 删除旧表（如果存在）
8  DROP TABLE IF EXISTS EMP;
9  DROP TABLE IF EXISTS DEPT;
10 DROP TABLE IF EXISTS SALGRADE;
11
12 -- 部门表
13 CREATE TABLE DEPT (
14     DEPTNO INT PRIMARY KEY COMMENT '部门编号',
15     DNAME VARCHAR(50) NOT NULL COMMENT '部门名称',
16     LOC VARCHAR(50) COMMENT '所在地'
17 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='部门表';
18
19 -- 工资等级表
20 CREATE TABLE SALGRADE (
21     GRADE INT PRIMARY KEY COMMENT '等级',
22     LOSAL DECIMAL(10,2) COMMENT '最低工资',
23     HISAL DECIMAL(10,2) COMMENT '最高工资'
24 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='工资等级
25 表';
26
27 -- 员工表
28 CREATE TABLE EMP (
29     EMPNO INT PRIMARY KEY COMMENT '员工编号',
30     ENAME VARCHAR(50) NOT NULL COMMENT '员工姓名',
31     JOB VARCHAR(50) COMMENT '职位',
32     MGR INT COMMENT '上级编号',
33     HIREDATE DATE COMMENT '入职日期',
34     SAL DECIMAL(10,2) COMMENT '月薪',
35     COMM DECIMAL(10,2) COMMENT '奖金',
36     DEPTNO INT COMMENT '部门编号',
37     FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO)
38 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='员工表';
```

```
39  -- 插入部门数据
40  INSERT INTO DEPT (DEPTNO, DNAME, LOC) VALUES
41  (10, 'ACCOUNTING', 'NEW YORK'),
42  (20, 'RESEARCH', 'DALLAS'),
43  (30, 'SALES', 'CHICAGO'),
44  (40, 'OPERATIONS', 'BOSTON');
45
46  -- 插入工资等级数据
47  INSERT INTO SALGRADE (GRADE, LOSAL, HISAL) VALUES
48  (1, 700, 11.0),
49  (2, 11.1, 1400),
50  (3, 1401, 2000),
51  (4, 2001, 3000),
52  (5, 3001, 9999);
53
54  -- 插入员工数据
55  INSERT INTO EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL,
56  COMM, DEPTNO) VALUES
57  (7369, 'SMITH', 'CLERK', 7902, '1980-11-17', 800,
58  NULL, 20),
59  (7499, 'ALLEN', 'SALESMAN', 7698, '1981-02-20', 1600,
60  300, 30),
61  (7521, 'WARD', 'SALESMAN', 7698, '1981-02-22', 11.0,
62  500, 30),
63  (7566, 'JONES', 'MANAGER', 7839, '1981-04-02', 2975,
64  NULL, 20),
65  (7654, 'MARTIN', 'SALESMAN', 7698, '1981-09-28', 11.0,
66  1400, 30),
67  (7698, 'BLAKE', 'MANAGER', 7839, '1981-05-01', 2850,
68  NULL, 30),
69  (7782, 'CLARK', 'MANAGER', 7839, '1981-06-09', 2450,
70  NULL, 10),
71  (7788, 'SCOTT', 'ANALYST', 7566, '1987-04-19', 3000,
72  NULL, 20),
73  (7839, 'KING', 'PRESIDENT', NULL, '1981-11-17', 5000,
74  NULL, 10),
```

```
65  (7844, 'TURNER', 'SALESMAN', 7698, '1981-09-08', 1500,
    0, 30),
66  (7876, 'ADAMS', 'CLERK', 7788, '1987-05-23', 1100,
    NULL, 20),
67  (7900, 'JAMES', 'CLERK', 7698, '1981-11.03', 950,
    NULL, 30),
68  (7902, 'FORD', 'ANALYST', 7566, '1981-11.03', 3000,
    NULL, 20),
69  (7934, 'MILLER', 'CLERK', 7782, '1982-01-23', 1300,
    NULL, 10);
```

所有表和数据准备就绪，让我们开始吧！

1. DQL 概述

DQL (Data Query Language) 主要负责从表中检索数据，核心命令是 **SELECT**。一条完整的 **SELECT** 语句包含多个子句，MySQL 内部大致按照以下顺序处理：



这个顺序非常重要，它解释了为什么 **WHERE** 中不能使用聚合函数，而 **HAVING** 可以。

2. 简单查询

2.1 查询所有列

```
1 | SELECT * FROM EMP;
```

2.2 查询指定列

```
1 | SELECT ENAME, JOB, SAL FROM EMP;
```

2.3 使用别名 (AS)

为列或表达式起一个更易读的名字：

```
1 | SELECT ENAME AS 姓名, SAL AS 月薪 FROM EMP;
```

2.4 表达式计算

可以在 SELECT 中对数值列进行计算：

```
1 | SELECT ENAME, SAL, SAL*11. AS 年薪 FROM EMP;
```

2.5 常量列

可以添加一个常量列：

```
1 | SELECT ENAME, '员工' AS 身份 FROM EMP;
```

3. 条件查询

3.1 比较运算符

=, <> (或 !=), >, <, >=, <=

查询月薪大于 2000 的员工：

```
1 | SELECT ENAME, SAL FROM EMP WHERE SAL > 2000;
```

查询职位不是 'SALESMAN' 的员工：

```
1 | SELECT ENAME, JOB FROM EMP WHERE JOB <> 'SALESMAN';
```

3.2 逻辑运算符

AND, OR, NOT

查询月薪在 1500 到 3000 之间的员工（使用 AND）：

```
1 | SELECT ENAME, SAL FROM EMP WHERE SAL >= 1500 AND SAL <= 3000;
```

查询在部门 10 或部门 20 的员工：

```
1 | SELECT ENAME, DEPTNO FROM EMP WHERE DEPTNO = 10 OR DEPTNO = 20;
```

NOT 可以取反，例如不是部门 30 的员工：

```
1 | SELECT ENAME, DEPTNO FROM EMP WHERE NOT DEPTNO = 30;
```

3.3 范围判断 BETWEEN AND

BETWEEN a AND b 包含边界值，等价于 $\geq a$ AND $\leq b$ 。

```
1 | SELECT ENAME, SAL FROM EMP WHERE SAL BETWEEN 1500 AND 3000;
```

注意：BETWEEN 也可以用于日期：

```
1 | SELECT ENAME, HIREDATE FROM EMP
2 | WHERE HIREDATE BETWEEN '1981-01-01' AND '1981-11.31';
```

3.4 IN 列表匹配

IN 用来判断某个值是否在列表中。

查询职位是 'CLERK' 或 'MANAGER' 的员工：

```
1 | SELECT ENAME, JOB FROM EMP WHERE JOB IN ('CLERK',  
      'MANAGER');
```

也可以用 `NOT IN` 排除这些职位。

3.5 空值判断 IS NULL / IS NOT NULL

判断奖金 (COMM) 是否为空：

```
1 | -- 没有奖金的员工  
2 | SELECT ENAME, COMM FROM EMP WHERE COMM IS NULL;  
3 |  
4 | -- 有奖金的员工  
5 | SELECT ENAME, COMM FROM EMP WHERE COMM IS NOT NULL;
```

`NULL` 不能用 `= NULL` 判断，必须用 `IS NULL`。

3.6 模糊查询 LIKE

`LIKE` 配合通配符 `%`（任意多个字符）和 `_`（任意一个字符）。

查询姓名以 'S' 开头的员工：

```
1 | SELECT ENAME FROM EMP WHERE ENAME LIKE 'S%';
```

查询姓名第二个字母是 'L' 的员工：

```
1 | SELECT ENAME FROM EMP WHERE ENAME LIKE '_L%';
```

查询姓名包含 'AR' 的员工：

```
1 | SELECT ENAME FROM EMP WHERE ENAME LIKE '%AR%';
```

4. 聚合函数

聚合函数对一组值进行计算，返回单个值。常用函数：

函数	作用
COUNT()	计数
SUM()	求和
AVG()	平均值
MAX()	最大值
MIN()	最小值

sql

```
1  -- 员工总数
2  SELECT COUNT(*) FROM EMP;
3
4  -- 有奖金的人数（COUNT(列名)会忽略NULL）
5  SELECT COUNT(COMM) FROM EMP;
6
7  -- 最高工资、最低工资、平均工资、工资总和
8  SELECT MAX(SAL), MIN(SAL), AVG(SAL), SUM(SAL) FROM EMP;
```

5. 分组过滤（GROUP BY 和 HAVING）

5.1 分组 GROUP BY

按部门统计每个部门的平均工资：

sql

```
1  SELECT DEPTNO, AVG(SAL) AS avg_sal
2  FROM EMP
3  GROUP BY DEPTNO;
```

5.2 带 WHERE 的分组

先筛选后分组：统计部门 10 和 20 的平均工资：

```
sql
1 SELECT DEPTNO, AVG(SAL) AS avg_sal
2 FROM EMP
3 WHERE DEPTNO IN (10,20)
4 GROUP BY DEPTNO;
```

5.3 HAVING 过滤分组结果

查询平均工资大于 2500 的部门：

```
sql
1 SELECT DEPTNO, AVG(SAL) AS avg_sal
2 FROM EMP
3 GROUP BY DEPTNO
4 HAVING AVG(SAL) > 2500;
```



注意：WHERE 在分组前筛选行，HAVING 在分组后筛选分组，不能混用。

6. 去重 DISTINCT

查看公司有哪些不同的职位：

```
sql
1 SELECT DISTINCT JOB FROM EMP;
```

查看哪些部门有员工（去重 DEPTNO）：

```
sql
1 SELECT DISTINCT DEPTNO FROM EMP;
```

也可以配合 COUNT 使用：统计有多少种不同的职位：

sql

```
1 | SELECT COUNT(DISTINCT JOB) FROM EMP;
```

7. 排序 ORDER BY

- **ASC** 升序（默认），**DESC** 降序。

按工资从高到低排序：

sql

```
1 | SELECT ENAME, SAL FROM EMP ORDER BY SAL DESC;
```

多列排序：先按部门编号升序，部门内再按工资降序：

sql

```
1 | SELECT DEPTNO, ENAME, SAL FROM EMP ORDER BY DEPTNO ASC, SAL  
DESC;
```

也可以使用别名排序：

sql

```
1 | SELECT ENAME, SAL*11.5 AS annual FROM EMP ORDER BY annual  
DESC;
```

8. 分页查询 LIMIT

MySQL 使用 **LIMIT** 限制结果行数。

返回前5条记录：

sql

```
1 | SELECT * FROM EMP LIMIT 5;
```

跳过前3条，取接下来的3条（常用于分页，比如第2页，每页3条）：

sql

```
1 | SELECT * FROM EMP LIMIT 3,3;
```

9. 子查询

子查询嵌套在另一个查询中，可以用在 **WHERE**、**FROM**、**SELECT** 等位置。

9.1 WHERE 中的子查询

查询工资比公司平均工资高的员工：

```
1 SELECT ENAME, SAL FROM EMP
2 WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

9.2 IN / NOT IN 与子查询

查询在部门所在地为 'NEW YORK' 或 'DALLAS' 的员工（需要关联子查询）：

```
1 SELECT ENAME, DEPTNO FROM EMP
2 WHERE DEPTNO IN (
3     SELECT DEPTNO FROM DEPT WHERE LOC IN ('NEW
4     YORK', 'DALLAS')
5 );
```

9.3 EXISTS / NOT EXISTS

查询有下属的员工（即有人把他当上级）：

```
1 SELECT ENAME FROM EMP e
2 WHERE EXISTS (
3     SELECT 1 FROM EMP WHERE MGR = e.EMPNO
4 );
```

9.4 FROM 子句中的子查询

必须给派生表起别名。查询每个部门的平均工资，并从中筛选平均工资大于2000的：

```
1 SELECT DEPTNO, avg_sal
2 FROM (
3     SELECT DEPTNO, AVG(SAL) AS avg_sal
4     FROM EMP
5     GROUP BY DEPTNO
6 ) AS dept_avg
7 WHERE avg_sal > 2000;
```

10. 多表联查

关系型数据库的精髓在于连接多个表查询。

10.1 内连接 INNER JOIN

查询员工姓名及其部门名称（只显示有部门的员工）：

```
1 SELECT e.ENAME, d.DNAME
2 FROM EMP e
3 INNER JOIN DEPT d ON e.DEPTNO = d.DEPTNO;
```

INNER JOIN 可简写为 JOIN。

10.2 左连接 LEFT JOIN

左表（EMP）全部保留，右表无匹配时显示 NULL。假设存在没有分配部门的员工，此处所有员工都有部门，但语法仍适用。

```
1 SELECT e.ENAME, d.DNAME
2 FROM EMP e
3 LEFT JOIN DEPT d ON e.DEPTNO = d.DEPTNO;
```

10.3 右连接 RIGHT JOIN

保留右表全部，实际中较少用，可通过交换表顺序用左连接实现。

10.4 多表连接（三表查询）

查询员工姓名、部门名称和工资等级：

```
sql
1  SELECT e.ENAME, d.DNAME, s.GRADE
2  FROM EMP e
3  JOIN DEPT d ON e.DEPTNO = d.DEPTNO
4  JOIN SALGRADE s ON e.SAL BETWEEN s.LOSAL AND s.HISAL;
```

注意连接条件可以是非等值连接（`BETWEEN`）。

10.5 自连接

查询每个员工及其上级的姓名：

```
sql
1  SELECT e1.ENAME AS 员工, e2.ENAME AS 上级
2  FROM EMP e1
3  JOIN EMP e2 ON e1.MGR = e2.EMPNO;
```

附录：MySQL 常用函数

1 字符串函数

函数	说明	示例
CONCAT(s1,s2,...)	连接字符串	<code>SELECT CONCAT(ENAME, '-', JOB) FROM EMP;</code>
UPPER(s)	转大写	<code>SELECT UPPER(ENAME) FROM EMP;</code>
LOWER(s)	转小写	<code>SELECT LOWER(ENAME) FROM EMP;</code>
SUBSTRING(s, start, len)	截取子串	<code>SELECT SUBSTRING(ENAME,1,3) FROM EMP;</code>
LENGTH(s)	字节长度	<code>SELECT LENGTH(ENAME) FROM EMP;</code>
CHAR_LENGTH(s)	字符数	<code>SELECT CHAR_LENGTH(ENAME) FROM EMP;</code>
REPLACE(s, from, to)	替换	<code>SELECT REPLACE(JOB, 'MANAGER', '经理') FROM EMP;</code>
TRIM(s)	去除首尾空格	<code>SELECT TRIM(ENAME) FROM EMP;</code>

2 数值函数

函数	说明	示例
ROUND(x,d)	四舍五入	<code>SELECT ROUND(SAL*1.1, 2) FROM EMP;</code>
CEIL(x)	向上取整	<code>SELECT CEIL(SAL) FROM EMP;</code>
FLOOR(x)	向下取整	<code>SELECT FLOOR(SAL) FROM EMP;</code>
MOD(x,y)	取余	<code>SELECT MOD(SAL,100) FROM EMP;</code>
RAND()	0~1 随机数	<code>SELECT RAND();</code>

3 日期函数

函数	说明	示例
NOW()	当前日期时间	<code>SELECT NOW();</code>
CURDATE()	当前日期	<code>SELECT CURDATE();</code>
DATE_FORMAT(d,fmt)	格式化日期	<code>SELECT DATE_FORMAT(HIREDATE, '%Y-%m-%d') FROM EMP;</code>
DATEDIFF(d1,d2)	日期差（天数）	<code>SELECT DATEDIFF(NOW(), HIREDATE) FROM EMP;</code>
YEAR(d)	提取年份	<code>SELECT YEAR(HIREDATE) FROM EMP;</code>
MONTH(d)	提取月份	<code>SELECT MONTH(HIREDATE) FROM EMP;</code>

4 流程控制函数

函数	说明	示例
IF(expr, v1, v2)	条件选择	<code>SELECT IF(SAL>2500, '高薪', '低薪') FROM EMP;</code>
CASE WHEN THEN ELSE END	多条件判断	见下文

sql

```
1  SELECT ENAME,  
2      CASE  
3          WHEN SAL >= 3000 THEN '高工资'  
4          WHEN SAL >= 1500 THEN '中等工资'  
5          ELSE '低工资'  
6      END AS 工资等级  
7  FROM EMP;
```

5 其他常用函数

- `COALESCE(expr1, expr2, ...)` 返回第一个非 NULL 值。可以用来处理奖金为空的情况：

```
SELECT ENAME, COALESCE(COMM, 0) FROM EMP;
```

- `IFNULL(expr1, expr2)` 是 `COALESCE` 的双参数版：`SELECT ENAME, IFNULL(COMM, 0) FROM EMP;`

- `DATABASE()` 返回当前数据库名。